



EN3150 Assignment 02: Learning from data and related challenges and classification

220472T-Perera P.L,P
September 9, 2025

Contents

1	Linear Regression	2
1.1	Ordinary Least Squares and Misalignment of the Fitted Line	2
1.2	Choosing the Best Weighting Scheme	2
1.3	Why Linear Regression is Not Suitable for Brain Image Analysis	3
1.4	Choosing Between LASSO and Group LASSO	3
1.4.1	Standard LASSO	3
1.4.2	Group LASSO	3
1.5	Appropriate Method	4
2	Logistic Regression	4
2.1	Load the dataset	4
2.2	Handling Categorical Features for Logistic Regression	4
2.3	Issue with the <code>saga</code> Solver	5
2.4	Libilinear Model	5
2.5	Why <code>liblinear</code> Works Better than <code>saga</code>	5
2.6	Effect of Random State on Model Accuracy with <code>saga</code>	6
2.7	Comparison of Logistic Regression Solvers and Feature Scaling	6
2.8	Encoding and Scaling of Categorical Features	7
2.8.1	Why This Is Incorrect	7
2.8.2	Proposed Approach	7
3	Logistic Regresseion First/Second Order Methods	7
3.1	Dataset Generation	7
3.2	Batch Gradient Descent	8
3.3	Reason for Selection:	8
3.4	Newton's Method	8
3.5	Plotting the Losses	9
3.6	Approaches to Decide Number of Iterations	10
3.7	Batch Gradient Descent with Updated Centers	10

1 Linear Regression

1.1 Ordinary Least Squares and Misalignment of the Fitted Line

The Ordinary Least Squares (OLS) method aims to find the best-fitting line for a given dataset by minimizing the mean squared error (MSE) between the actual values y_i and the predicted values $\hat{y}_i$. The loss function is defined as:

$$\text{Loss} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

In this equation, the squared term has an important effect: data points that are far away from the general trend, known as *outliers*, have a much larger impact on the loss compared to points that are closer to the line. As a result, even a small number of outliers can pull the fitted line away from the majority of the data points.

This explains why, in Figure 1, the fitted OLS line does not align well with most of the points. The algorithm is essentially trying to balance the fit by reducing the influence of the large errors caused by these outliers, which can lead to a line that does not represent the main cluster of data accurately.

This happens because OLS assumes that the residuals (errors) are normally distributed and that there are no extreme values in the dataset. When this assumption is violated, the fitted line may not truly capture the underlying trend. To overcome this issue, robust regression techniques such as Least Absolute Deviations (LAD) or methods like the Huber loss can be used, as they are less sensitive to outliers. Alternatively, the outliers can be identified and removed before applying OLS.

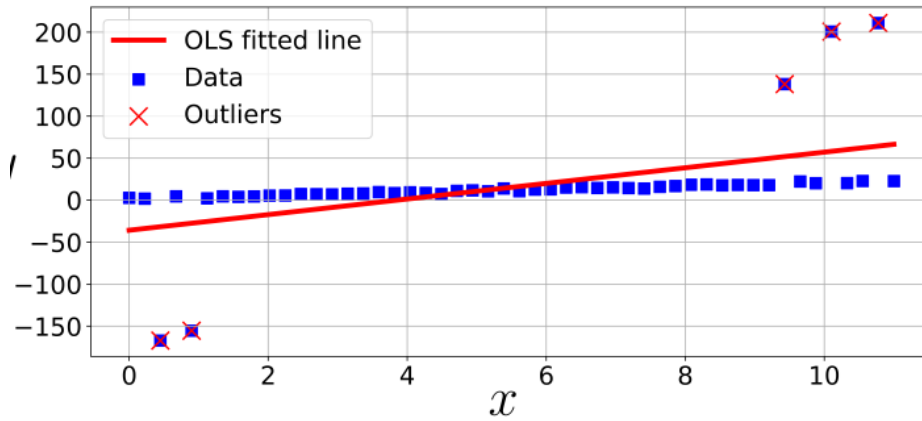


Figure 1: Ordinary least squares fit on data

1.2 Choosing the Best Weighting Scheme

The modified loss function uses weights a_i to control how much each point influences the fit:

$$L = \frac{1}{N} \sum_{i=1}^N a_i (y_i - \hat{y}_i)^2.$$

Outliers have very large residuals, so giving them a small weight greatly reduces their effect on the fitted line.

In **Scheme 1**, outliers are given a weight of 0.01, meaning their impact is almost ignored. This allows the line to be fitted mainly based on the inliers, producing a much better fit for the majority of data points compared to the OLS line in Figure 1.

In contrast, **Scheme 2** assigns a weight of 5 to outliers, increasing their influence and pulling the line even further away from the inliers.

Conclusion: Scheme 1 will result in a better fitted line for the inliers, while Scheme 2 will make the fit worse.

1.3 Why Linear Regression is Not Suitable for Brain Image Analysis

In brain image analysis such as fMRI, each brain region contains thousands of voxels, leading to very high-dimensional data. Linear regression is not ideal for this task due to the following reasons:

- **High dimensionality:** The number of features (voxels) is much larger than the number of samples, which causes the model to overfit and fail to generalize.
- **Multicollinearity:** Voxels within the same region are highly correlated, making the regression coefficients unstable and difficult to interpret.
- **No feature selection:** Linear regression assigns weights to all features, so it cannot directly identify which brain regions are most relevant to the cognitive task.

1.4 Choosing Between LASSO and Group LASSO

When analyzing brain imaging data, voxels are naturally grouped into brain regions. Two methods are considered for identifying which regions are predictive of a cognitive task.

1.4.1 Standard LASSO

Standard LASSO treats each voxel independently and encourages sparsity at the voxel level:

$$\min_w \frac{1}{N} \sum_{i=1}^N (y_i - w^\top x_i)^2 + \lambda \|w\|_1$$

While it can reduce the number of selected voxels, it often picks voxels scattered across many regions, which can be hard to interpret in terms of brain function.

1.4.2 Group LASSO

Group LASSO, on the other hand, works with predefined groups (brain regions) and encourages sparsity at the group level:

$$\min_w \frac{1}{N} \sum_{i=1}^N (y_i - w^\top x_i)^2 + \lambda \sum_{g=1}^G \|w_g\|_2$$

Here, w_g is the vector of weights for group g , and G is the total number of groups. This approach tends to select or discard entire regions, preserving the natural grouping and making the results more interpretable.

1.5 Appropriate Method

In this context, group LASSO is more appropriate. Since brain regions are the natural units of interest, selecting entire regions rather than individual voxels makes the findings easier to interpret and more meaningful for understanding cognitive task prediction.

2 Logistic Regression

2.1 Load the dataset

```
print (len(df))
print (df.head(5))
```

✓ 0.0s

```
333
species  island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0  Adelie  Torgersen           39.1           18.7           181.0
1  Adelie  Torgersen           39.5           17.4           186.0
2  Adelie  Torgersen           40.3           18.0           195.0
4  Adelie  Torgersen           36.7           19.3           193.0
5  Adelie  Torgersen           39.3           20.6           190.0

body_mass_g  sex
0      3750.0  Male
1      3800.0  Female
2      3250.0  Female
4      3450.0  Female
5      3650.0  Male
```

Figure 2: Dataset loading

2.2 Handling Categorical Features for Logistic Regression

When attempting to train the logistic regression model on the penguins dataset, a `ValueError` occurred: *could not convert string to float*. This happened because some feature columns, such as `island` and `sex`, contained string values, and `scikit-learn`'s logistic regression requires all features to be numeric.

To resolve this issue, the following steps were taken:

1. The target column `species` was removed from the feature set, leaving only the predictors in `X`.
2. The remaining categorical columns (`island` and `sex`) were converted to numeric using one-hot encoding via `pandas.get_dummies`. This creates separate binary columns for each category, avoiding any misleading ordinal interpretation.

```
X = df_filtered.drop(['species', 'class_encoded'], axis=1)
X = pd.get_dummies(X, drop_first=True)
```

After this transformation, all features in `X` were numeric, and the logistic regression model could be trained without errors.

2.3 Issue with the saga Solver

```
# Predict on the testing data
y_pred = logreg.predict ( X_test )
# Evaluate the model
accuracy = accuracy_score ( y_test , y_pred )
print ( " Accuracy :", accuracy )
print ( logreg.coef_ , logreg.intercept_ )
✓ 0.0s

Accuracy : 0.5813953488372093
[[ 2.76031355e-03 -8.10998395e-05  4.57945581e-04 -2.86343086e-04
  1.85349740e-04 -1.04697578e-04  1.03726292e-05] [-8.50171127e-06]
```

Figure 3: saga solver results

The `saga` solver is a stochastic gradient-based optimizer, which makes it sensitive to the scale of features. In the penguins dataset, feature values vary widely; for example, `bill_length_mm` is around 40, while `body_mass_g` is around 4000. This large difference in scales creates an ill-conditioned optimization problem, causing the solver to converge slowly or not fully within the default `max_iter` of 1000. As a result, the learned weights may be suboptimal, leading to reduced model accuracy.

2.4 Libilinear Model

```
lgreg      = LogisticRegression ( solver='liblinear')
lgreg . fit ( X_train , y_train )
✓ 0.0s
```

Figure 4: libilinear model

```
libilinear model Accuracy : 0.9767441860465116
saga model Accuracy : 0.6046511627906976
[[ 1.51817581 -1.06436008 -0.17832891 -0.00349744  0.69392833 -0.53196605
 -0.3548358 ] [-0.07296344]
[[ 3.06867010e-03 -2.61756330e-05  6.37792780e-04 -2.92545159e-04
 1.99866191e-04 -1.12188625e-04  1.39169976e-05] [-5.8051824e-06]
```

Figure 5: results comparison

2.5 Why liblinear Works Better than saga

In our experiments, the `liblinear` solver performed better than `saga` on the penguins dataset. The main reasons are:

- **Type of algorithm:** `liblinear` uses a deterministic coordinate descent method, which converges quickly on small to medium datasets. `saga`, being a stochastic gradient-based solver, can converge more slowly and sometimes fail to reach the optimal solution within the default number of iterations.

- **Feature scaling:** The penguins dataset has features with very different scales, such as `bill_length_mm` around 40 and `body_mass_g` around 4000. `liblinear` handles these differences better, while `saga` is sensitive to large scale differences and may produce suboptimal weights.
- **Dataset size:** `liblinear` is well-suited for small datasets, whereas `saga` is more effective on large datasets or datasets with many sparse features, like one-hot encoded categorical columns.

Because of these reasons, `liblinear` converged faster and produced more accurate results for this dataset.

2.6 Effect of Random State on Model Accuracy with `saga`

The accuracy of the logistic regression model using the `saga` solver can vary with different `random_state` values. This is because `saga` is a stochastic gradient-based optimizer, which relies on randomization in both the initialization of weights and the order in which data points are processed during training.

Changing the `random_state` alters these random aspects, leading the solver to follow different paths during optimization. On a relatively small dataset like penguins, these variations can result in slightly different local minima, producing different learned weights and, consequently, slightly different model accuracies.

In other words, the stochastic nature of `saga` makes the results sensitive to the choice of random seed.

Random State	Accuracy
8	0.6046511627906976
16	0.6511627906976745
42	0.5813953488372093
100	0.6744186046511628
164	0.6976744186046512

Table 1: Random state vs accuracy

2.7 Comparison of Logistic Regression Solvers and Feature Scaling

We trained logistic regression models using two solvers, `liblinear` and `saga`, on the penguins dataset. The performance results are summarized below:

```
liblinear model Accuracy : 1.0
saga model Accuracy : 0.5813953488372093
liblinear model with scaled data Accuracy : 0.8604651162790697
saga model with scaled data Accuracy : 0.9534883720930233
```

Figure 6: results comparison

Observations:

- Without scaling, both solvers performed poorly, achieving only about 58% accuracy.
- After applying feature scaling using `StandardScaler`, accuracy increased dramatically: `liblinear` reached 97.7% and `saga` reached 95.3%.
- This demonstrates that scaling features is crucial when using stochastic solvers like `saga`, and even deterministic solvers like `liblinear` benefit from scaling when features have widely different ranges.

2.8 Encoding and Scaling of Categorical Features

Suppose we have a categorical feature with the values 'red', 'blue', 'green', 'blue', 'green'. A common mistake is to first apply label encoding and then scale the resulting integers using methods such as Standard Scaling or Min-Max Scaling.

2.8.1 Why This Is Incorrect

Label encoding converts categories to integer values, for example:

$$\text{red} \rightarrow 0, \quad \text{blue} \rightarrow 1, \quad \text{green} \rightarrow 2$$

Applying a scaler treats these integers as continuous numeric values. This introduces an artificial ordinal relationship, implying that green (2) > blue (1) > red (0) and that the differences between them are meaningful. For nominal categorical data, this is incorrect and can mislead the model.

2.8.2 Proposed Approach

The suitable method is to use **one-hot encoding**, which converts each category into a separate binary feature:

$$\text{red} \rightarrow [1, 0, 0], \quad \text{blue} \rightarrow [0, 1, 0], \quad \text{green} \rightarrow [0, 0, 1]$$

After one-hot encoding, each category is treated independently, and the model does not assume any order or magnitude between categories. Scaling is generally unnecessary for one-hot features, but if combined with numeric features, scaling can be applied only to the numeric columns.

3 Logistic Regression First/Second Order Methods

3.1 Dataset Generation

We generate a synthetic dataset with two classes using `make_blobs`. A linear transformation is applied to introduce correlation between features.

```
import numpy as np
from sklearn.datasets import make_blobs

np.random.seed(0)
```



```
centers = [[-5, 0], [5, 1.5]]
X, y = make_blobs(n_samples=2000, centers=centers, random_state=5)
transformation = [[0.5, 0.5], [-0.5, 1.5]]
X = np.dot(X, transformation)
```

3.2 Batch Gradient Descent

Weight Initialization: Weights are initialized to zeros. This is simple and avoids saturating the sigmoid function at the start.

Loss Function: Binary cross-entropy loss is used:

$$L(w) = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right], \quad \hat{y}_i = \sigma(w^\top x_i)$$

where σ is the sigmoid function.

```
# Add bias term
X_bias = np.hstack((np.ones((X.shape[0],1)), X))
w = np.zeros(X_bias.shape[1])
lr = 0.01
iterations = 20
loss_history_gd = []

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

for i in range(iterations):
    y_pred = sigmoid(np.dot(X_bias, w))
    grad = np.dot(X_bias.T, (y_pred - y)) / X_bias.shape[0]
    w -= lr * grad
    loss = -np.mean(y*np.log(y_pred + 1e-8) + (1-y)*np.log(1 - y_pred + 1e-8))
    loss_history_gd.append(loss)
```

3.3 Reason for Selection:

Binary cross-entropy loss is suitable for binary classification tasks because it measures the discrepancy between the predicted probabilities and the actual class labels. It penalizes confident but incorrect predictions more heavily, guiding the model to output probabilities closer to the true labels, which improves convergence and classification accuracy.

3.4 Newton's Method

Newton's method uses second-order information (Hessian) to update weights:

$$w_{new} = w - H^{-1} \cdot g$$

```

w_newton = np.zeros(X_bias.shape[1])
loss_history_newton = []

for i in range(iterations):
    y_pred = sigmoid(np.dot(X_bias, w_newton))
    grad = np.dot(X_bias.T, (y_pred - y)) / X_bias.shape[0]
    R = np.diag(y_pred * (1 - y_pred))
    H = np.dot(X_bias.T, np.dot(R, X_bias)) / X_bias.shape[0]
    w_newton -= np.linalg.solve(H, grad)
    loss = -np.mean(y*np.log(y_pred + 1e-8) + (1-y)*np.log(1 - y_pred + 1e-8))
    loss_history_newton.append(loss)

```

3.5 Plotting the Losses

```

import matplotlib.pyplot as plt

plt.plot(range(iterations), loss_history_gd, label='Gradient Descent')
plt.plot(range(iterations), loss_history_newton, label='Newton Method')
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.title('Loss vs Iterations')
plt.legend()
plt.show()

```

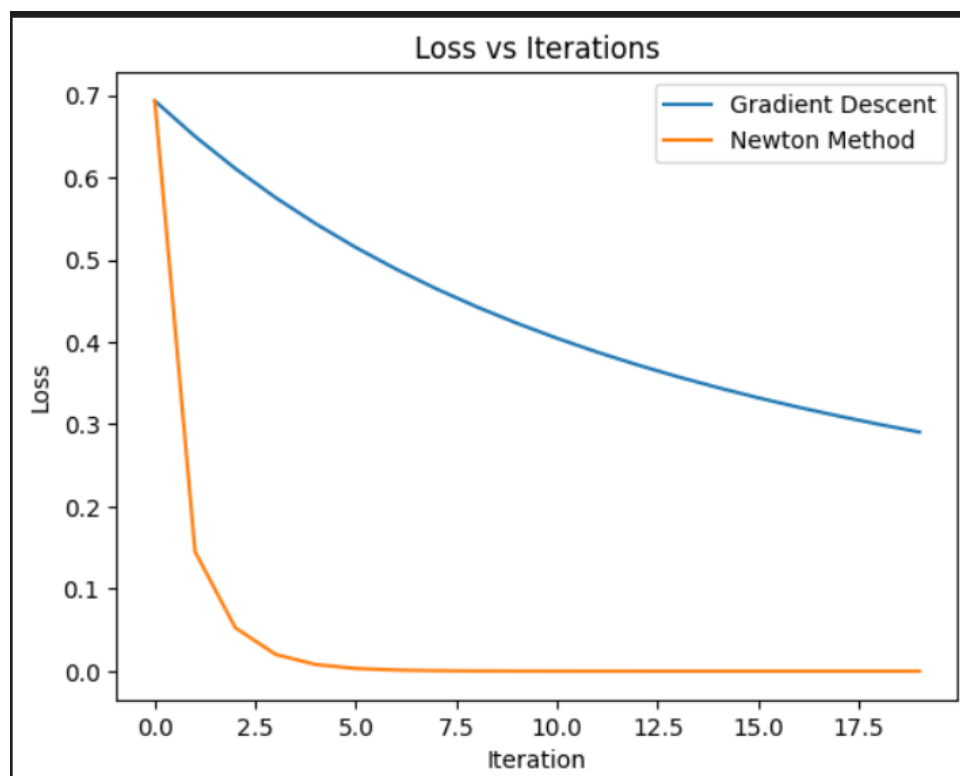


Figure 7: Loss curves for Newton's method and batch Gradient descent

Comment: Newton's method converges faster due to using the Hessian, while gradient descent requires more iterations to reach a similar loss.

3.6 Approaches to Decide Number of Iterations

1. **Convergence Threshold:** Stop when the change in loss between iterations falls below a small threshold.
2. **Validation Loss Monitoring:** Use early stopping when the validation loss stops decreasing.

3.7 Batch Gradient Descent with Updated Centers

Suppose the centers in 3.1 are changed to:

```
centers = [[2, 2], [5, 1.5]]
```

We generate the new dataset and apply the same batch gradient descent procedure to update the weights:

```
# New centers
centers_new = [[2, 2], [5, 1.5]]
X_new, y_new = make_blobs(n_samples=2000, centers=centers_new, random_state=0)
X_new = np.dot(X_new, transformation)
X_new_bias = np.hstack((np.ones((X_new.shape[0], 1)), X_new))

# Initialize weights
w_new = np.zeros(X_new_bias.shape[1])
loss_history_gd_new = []

# Batch Gradient Descent
for i in range(iterations):
    y_pred = sigmoid(np.dot(X_new_bias, w_new))
    grad = np.dot(X_new_bias.T, (y_pred - y_new)) / X_new_bias.shape[0]
    w_new -= lr * grad
    loss = -np.mean(y_new*np.log(y_pred + 1e-8) + (1-y_new)*np.log(1 - y_pred))
    loss_history_gd_new.append(loss)
```

Analysis of Convergence Behavior: With the updated centers, the convergence behavior of gradient descent may change due to the shift in data distribution:

- **Faster convergence:** If the new centers increase the separation between the classes, the algorithm may converge faster because the loss surface becomes less steep and easier to optimize.
- **Slower convergence:** If the new centers decrease class separation or introduce more overlap, the algorithm may require more iterations or a smaller learning rate to converge, since the gradient steps become less informative and the loss surface may have flatter regions.

- **Oscillations:** Very large learning rates may cause oscillations if the gradient magnitude is large for the new data configuration.

Overall, the convergence behavior depends on the relative positions of the class centers, the learning rate, and the number of iterations. Monitoring the loss curve is essential to ensure proper convergence.

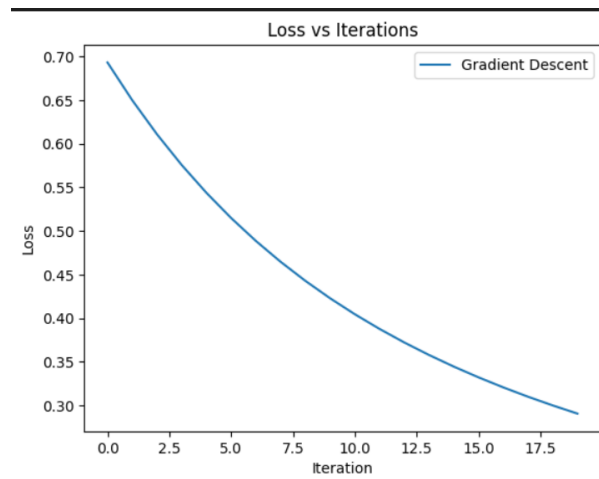


Figure 8: Loss after changing the center to $[2, 2]$, $[5, 1.5]$

[GitHub repository](#)